

The background of the cover is composed of several large, overlapping geometric shapes. On the right side, there is a large blue chevron pointing downwards. On the left side, there is a large, light gray chevron pointing upwards. These shapes create a dynamic, modern aesthetic.

PROCESSO SELETIVO WATT
AUTOMAÇÃO INDUSTRIAL E SUPERVISÃO

1 Apresentação

Este desafio tem como objetivo avaliar a capacidade do candidato de projetar e implementar uma solução de engenharia de ponta a ponta, integrando aquisição de dados em tempo real, persistência estruturada, exposição industrial e supervisão. O cenário simula uma aplicação real de monitoramento de qualidade de energia e exige decisões técnicas bem fundamentadas, documentação clara e evidências objetivas de funcionamento.

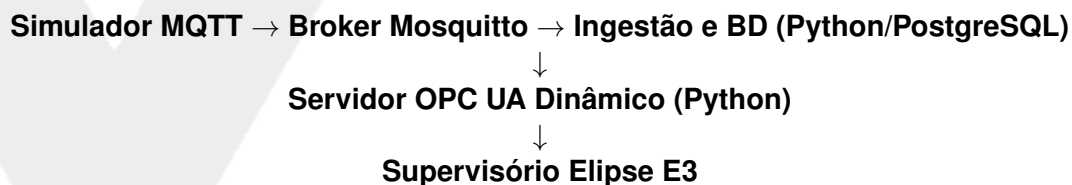
Leia o enunciado na íntegra antes de começar. Cada bloco do escopo obrigatório depende dos anteriores; entender o fluxo completo desde o início reduz retrabalho e melhora a consistência da solução final.

2 Contexto do problema

Considere o desenvolvimento de uma plataforma de monitoramento de energia elétrica em tempo real para uma instalação industrial. O objetivo é monitorar a energia que chega da concessionária na subestação principal e comparar com o comportamento de duas máquinas específicas no chão de fábrica, que possuem características elétricas muito distintas.

Os dados devem ser gerados continuamente por um simulador e transmitidos via um broker MQTT (Mosquitto). A solução deve ser capaz de assinar esses tópicos, normalizar os dados, persisti-los de forma estruturada em um banco de dados e disponibilizá-los sob demanda de contexto para o supervísório Elipse E3, atuando como um servidor OPC UA.

O fluxo geral esperado é o seguinte:



3 Especificação da simulação e dados de entrada

O candidato deve construir um simulador (script independente) que publique dados no broker MQTT a uma taxa de atualização de 1 segundo.

Importante para testes: O simulador deve possuir um mecanismo de injeção de falhas (seja por atalhos de teclado via terminal, interface CLI, botões web ou comandos MQTT) que permita forçar manualmente as anomalias elétricas descritas abaixo. Isso será exigido no dia da apresentação para testar a resposta do sistema.

As máquinas simuladas devem respeitar as características elétricas descritas abaixo:

3.1 Ponto 1: Subestação Principal (Entrada da Rede)

Representa a energia entregue pela concessionária, devendo publicar:

- **Tensão (V):** Deve variar levemente em torno de 380V. O simulador deve gerar afundamentos de tensão (quedas repentinas para 350V) aleatórios ou sob comando.
- **Potência Ativa Total (kW):** Soma do consumo da fábrica.
- **Fator de Potência Global:** Deve se manter próximo a 0.95.

3.2 Ponto 2: Máquina 1 - Compressor de Ar Parafuso

Carga altamente indutiva com picos de partida severos.

- **Corrente (A):** Simular operação em ciclos (liga/desliga). Ao ligar, simular um pico de corrente (*inrush*) de 5 a 7 vezes a corrente nominal por alguns segundos.
- **Fator de Potência:** Cai drasticamente (ex: 0.70) durante o pico de partida, estabilizando em 0.88.

3.3 Ponto 3: Máquina 2 - Extrusora Plástica

Equipamento acionado por inversor de frequência que gera poluição na rede.

- **THD de Corrente (%):** Distorção harmônica total com valores altos (entre 15% e 25%).
- **Temperatura do Pannel (°C):** Varia gradativamente com o aumento da distorção harmônica e carga.

4 Escopo obrigatório

A solução é composta por cinco blocos funcionais interdependentes. Cada bloco deve ser implementado, documentado e ter seu funcionamento evidenciado na entrega.

4.1 Bloco 1 - Ingestão de dados em tempo real

Implemente um serviço (*middleware* em Python) que assine os tópicos do broker MQTT e consuma as mensagens continuamente. O serviço deve:

- Receber mensagens de forma contínua, sem bloquear o fluxo por falhas individuais.
- Validar a estrutura e os tipos de cada grandeza recebida.
- Registrar logs de erro e de processamento com nível de detalhe suficiente para diagnóstico.

4.2 Bloco 2 - Persistência em banco de dados

Persista as leituras em um banco de dados relacional, obrigatoriamente **PostgreSQL**. A modelagem deve contemplar no mínimo:

- Tabela de ativos (cadastro das máquinas e subestação).
- Tabela de leituras históricas (séries temporais), associadas ao ativo e ao instante de medição.

O script Python não deve perder dados caso o banco de dados fique indisponível brevemente, devendo implementar um mecanismo de *buffer* ou fila local temporária.

4.3 Bloco 3 - Disponibilização e Servidor OPC UA Dinâmico

Esta etapa é o núcleo de integração. O *middleware* Python deve atuar como um servidor **OPC UA**. No entanto, o fornecimento de dados **não** deve ser feito mapeando variáveis simples (1 por 1).

- O Eclipse E3 deverá escrever em uma *tag* OPC UA chamada *ActiveScreenID*, indicando qual tela o operador está visualizando (ex: TelaSubestacao, TelaCompressor).

- Ao detectar a mudança desta *tag*, o servidor Python deve buscar os dados históricos correspondentes àquela máquina no PostgreSQL e empacotá-los em um **Array (vetor de dados)** dinâmico.
- O Array é então disponibilizado via OPC UA para que o E3 apenas o consuma e plote os gráficos, transferindo a carga de processamento do banco para o Python.

4.4 Bloco 4 - Supervisório em Elipse E3

Implemente o supervisório utilizando a versão de demonstração do Elipse E3. O projeto deve demonstrar:

- **Tela Principal:** *Dashboard* de visão geral exibindo o status das máquinas e a potência total.
- **Telas Específicas:** Uma tela para cada ativo, exibindo os *Arrays* recebidos do Python em formato gráfico.
- **Scripts Internos:** Utilização de VBScript internamente **apenas** para controle de interface e leitura dos Arrays, delegando regras de negócio ao servidor Python.

4.5 Bloco 5 - Interpretação dos dados e Alarmes

A plataforma deve gerar interpretações técnicas a partir dos dados brutos configurando alarmes no Elipse E3. As interpretações mínimas obrigatórias são:

- **Afundamento de Tensão:** Alarme acionado quando a tensão da subestação cair drasticamente.
- **Partida Severa:** Detecção do pico de corrente anômalo prolongado no compressor.
- **Poluição Harmônica Crítica:** Alarme acionado quando a THD da extrusora ultrapassar limites seguros (ex: > 20%).

5 Entregáveis

A entrega deve ser feita em repositório organizado no GitHub e deve conter os seguintes itens:

Item	Descrição
Código-fonte	Código completo do simulador MQTT e do servidor Python (OPC UA + BD), com comentários nas partes relevantes.
README.md	Instruções reproduzíveis para configurar e executar o projeto do zero (recomenda-se o uso de <code>docker-compose</code>).
Schema do banco	Arquivo <code>.sql</code> com a definição das tabelas ou coleções criadas.
Projeto Elipse E3	Arquivos <code>.prq</code> e <code>.dom</code> compactados.

6 Requisitos técnicos

- **Linguagem principal:** Python para o middleware/servidor OPC UA e para o simulador MQTT.

- **Banco de dados:** PostgreSQL.
- **Organização do código:** Separação mínima por módulos com responsabilidades bem definidas.
- **Tratamento de falhas:** Falhas em componentes individuais não devem derrubar o serviço.
- **Reprodutibilidade:** Qualquer avaliador deve conseguir executar o projeto seguindo apenas o README.md.

7 Apresentação final

Na apresentação, o candidato deve demonstrar ao vivo ou por evidências gravadas (vídeo):

1. O simulador rodando e publicando dados de perfis distintos no Mosquitto.
2. A ingestão e gravação robusta no PostgreSQL.
3. A mudança da tag `ActiveScreenID` no E3 alterando dinamicamente a query no Python e o Array fornecido via OPC UA.
4. O supervisor Elipse E3 consumindo esses arrays e desenhando os gráficos sem consultas nativas pesadas a bancos de dados.
5. O acionamento correto dos alarmes baseados nos comportamentos elétricos das máquinas.
6. A injeção de falhas manuais no simulador para demonstrar o sistema reagindo e alarmando em tempo real.

O candidato deve estar preparado para explicar as decisões de arquitetura adotadas e responder perguntas sobre qualquer parte da solução.



WATT